Git-Vista Security Model

Status: living document — the V1 model is largely implemented; sections marked *future* or *V2* are not. The per-item `*(Implemented: …)*` annotations and the ADRs they cite are authoritative for what has actually shipped. **This banner is not** — it said "proposed for V2" for the whole of M1.13b, while the body already recorded the sandbox enforcement as shipped (ADR 0030). Read the annotations, not the header.

Git-Vista is local-first and primarily single-user, but it controls real Git repositories and can execute destructive operations. "Only for me" reduces the identity problem; it does not remove browser-origin attacks, stale tabs, malicious LAN clients, credential leakage, or command races. Repository *content* is also untrusted input: names, messages, refs and paths are treated as hostile and are validated. Repository *code* — hooks, filters, and any config key that names an executable — is a different matter. It runs under bounded, irreversible kernel restrictions, tiered by operation kind: Landlock filesystem rules wherever repository code runs, Landlock network rules that scope the network tier to a fixed TCP port list — never to a destination host, see ADR 0028 — seccomp syscall filtering, and, outside the network tier, a `bwrap` namespace boundary. **This enforcement has shipped (ADR 0030).** Every local git operation the server spawns for an untrusted repository now runs inside `Tier::Strict` or `Tier::Network` — bwrap namespaces, then Landlock, then seccomp, applied by the `gv-sandbox` shim after `exec`, reached through a sealed argv chokepoint a caller cannot append to. `Tier::Unsandboxed` exists for operator-trusted repositories only and is reachable by rule, not yet by any handler-facing route. What a session's hooks actually run under is disclosed per repository as one of four named values — `Strict` / `Network` / `Unsandboxed` / `Blocked` — matching the real tiers directly, not the old `Allow` / `Restricted` labels (ADR 0025, amended by ADR 0030); an undisclosed policy is the field's *absence*, never a value that claims more than was measured. It does not claim isolation from a same-uid adversary, and it cannot: see Known Non-Goals and Sandbox Mechanism Boundaries below for exactly what each layer covers and where it does not.

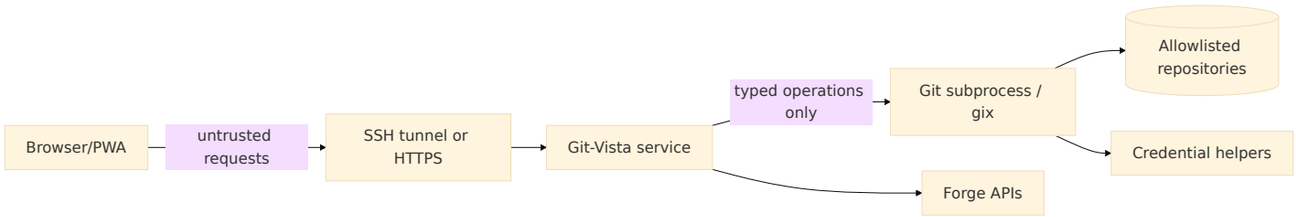
Security Objectives

- A website opened in the same browser cannot command the local Git-Vista server.
- A LAN device cannot discover and mutate repositories without explicit pairing.
- No browser or API request can directly supply arbitrary paths, Git argv, environment variables, or a shell — enforced at the browser/API surface (ADR 0017). Repository code itself — hooks, filters, configured executables — has its own execution authority once Git invokes it, and is governed separately; see Command Execution and Known Non-Goals.
- A stale or replayed request cannot repeat a mutation silently.
- Provider and Git credentials never enter logs, URLs, API payloads, or browser storage unnecessarily.
- Git-Vista's own read paths cap memory use, output size, and process lifetime — quantified ceilings on what Git-Vista itself spawns, not a blanket bound on the repository. *(Implemented for history and file reads: ADR 0022, #63 — every git read goes through `git_vista_git::git_stdout_capped`, which streams under a per-kind cap (8 MiB diff metadata → 413, 200 KB per patch within 5 MB, 2 MB per file) and carries `kill_on_drop`, so a disconnected client kills the child instead of letting it finish into a buffer. History is paged rather than buffered whole, and paging keeps **no** per-client server state: the entire state of a scroll is one signed offset in the client's cursor, so memory is independent of both repository size and the number of connected clients. The frontend culls to at most 2,000 live rows regardless of camera. The memory bound is a denial-of-service control, not merely a performance measure. **Disk use and a hook's own child processes are not bounded by any of this** — repository code executes as the real uid (ADR 0025) with no ceiling of its own, and same-uid resource exhaustion is a Known Non-Goal, not a gap in this control.)*
- Recovery data is protected at least as strongly as the repository itself.
- Security remains understandable for one person to operate.

Assets

- Repository objects, refs, worktrees, index, stashes, and uncommitted changes.
- SSH keys, Git credential-helper material, forge OAuth tokens, and cookies.
- Operation history and recovery refs.
- Repository paths and file contents.
- The ability to push, force-push, create PRs, and modify remote state.

Trust Boundaries



The browser is not trusted merely because it served the UI. Every mutation goes through authentication, origin validation, repository authorization, generation checks, operation planning, and the per-worktree queue.

Operating Modes

Mode	Bind	Transport	Authentication	Intended use
Local	`127.0.0.1`	HTTP localhost	Launch/session secret + same-origin	Browser on the Linux/macOS/Windows host
SSH tunnel	`127.0.0.1` on Linux	SSH encrypted forwarding	SSH plus Git-Vista session	Primary iPad-to-Linux workflow
LAN view	One explicit interface	Plain HTTP	Single-use bootstrap token, view-scoped read-only routes, rate-limited sign-in	Backup path when the SSH tunnel is unavailable, trusted home LAN only (ADR 0005)
LAN paired, future	Explicit interface	HTTPS	One-time pairing and device session	Trusted private network without SSH tunnel, full read/write
Team, future	Reverse proxy/private network	HTTPS	OIDC/passkeys plus RBAC	Explicit multi-user deployment, not V2 default

Modes are configuration profiles, not a boolean `--public` switch. Each profile has different startup checks and refuses to start when required controls are absent.

Local and SSH Session Design

Implemented by the session + request-protection layer (M1.04, `git-vista-server::{session,security}`; ADR 0004). The bootstrap token is written `0600` and delivered via the `gv` setup link's URL *fragment* (never the server, never a log); everything below is enforced by the `require_auth` layer.

- Generate a high-entropy session secret at every service start unless a durable paired-device session is explicitly configured. *(256-bit `getrandom` token per start; the in-memory store means a restart is a full revocation. Durable paired sessions deferred to the LAN/paired milestone.)*
- Print/open a bootstrap URL whose one-time secret is exchanged for an HttpOnly, SameSite=Strict session cookie. Remove secrets from the visible URL immediately. *(`gv` prints `…/#s=<token>`; the SPA `POST`s it and strips the fragment with `history.replaceState`. The token is single-use — redeeming it rotates a fresh one — and expires.)*
- Require a separate CSRF token on every state-changing request. *(Per-session token echoed in `x-git-vista-csrf`, compared constant-time; missing/invalid on a live session is a `403`.)*
- Validate `Origin`, `Host`, and content type. Reject `Origin: null` mutations. *(Host must be a loopback literal — the anti-DNS-rebinding check; `Origin`, when present, must be same-origin and non-`null`; a present content type on a write must be JSON, blocking form-encoded CSRF.)*
- Set a strict Content Security Policy and deny framing with `frame-ancestors 'none'`. *(See "Browser Security Headers" — stamped on every response by `security_headers`.)*

- Bind only to loopback in Local and SSH modes. *(Default bind is `127.0.0.1`; the Host/Origin policy is derived from the bind address.)*
- Treat localhost as vulnerable to malicious webpages and DNS rebinding; binding loopback is necessary but not sufficient. *(The reason the Host allowlist and session gate exist at all.)*
- Expire operation approval tokens quickly and bind them to session, repository, worktree, operation hash, and repository generation. *(Session/bootstrap expiry landed here; per-operation approval tokens are a later milestone.)*

LAN View Profile (implemented, ADR 0005)

`gv --lan-view [path]` starts the existing loopback server plus a second listener, bound to one explicit, operator-confirmed LAN IP:

- The second listener serves a structurally reduced router: GET read routes plus `POST` / `DELETE` `/api/session` only. Every write, `/api/select`, `/api/rescan`, `/api/clone`, and `/api/delete-clone` route is never registered on it — absence, not a runtime check (`crates/git-vista-server/ src/main.rs::api_router`).
- `Host` / `Origin` on this listener are pinned to the one sanctioned LAN IP:port (`security::HostPolicy::lan`); neither `localhost` nor any other address the machine answers on is accepted, so a DNS-rebinding attempt against the LAN listener fails closed the same way the loopback listener's Host check does.
- Sign-in (`POST /api/session`) on this listener is rate-limited per source IP (`crates/git-vista-server/src/ratelimit.rs`); the loopback listener's sign-in is unaffected.
- Auth is otherwise the same single-use bootstrap-token flow as loopback, sharing one in-memory session store; a session established via either listener carries a `via_lan` flag purely so the UI can hide the Active option — the actual write boundary is the LAN router's absent routes. *(Implemented: ADR 0024, #64 — the frontend's CSRF token and `via_lan` flag, formerly two independent `thread_local!`s with no rule tying them together, now live together in one `SessionCore` with a typed `SessionRejection::UiModeChangeWhileLan`, so "a LAN-view session may not select Active mode" is answerable from one place. This is a client-side UI affordance only, reinforcing — not replacing — the write boundary above, which remains the LAN router's absent routes.)*
- Accepted, documented risk: plain HTTP means repo contents and the session cookie are readable by anyone on the same network. Suitable for a trusted home LAN, never a guest or shared network — the startup banner and `gv doctor` say so explicitly.
- `gv doctor` and the launch-time exposed-listener kill-check learn the sanctioned second socket: with `--lan-view`, exactly {loopback, the recorded LAN ip} on port 8080 is healthy; anything else is still a SECURITY ERROR that stops the server. Without the flag, behavior is unchanged from M1.05.

LAN Mode (future, paired HTTPS — write-capable)

The read-only LAN view profile above (ADR 0005) is what's implemented today; this section covers the *different*, still-future write-capable LAN mode — the "LAN paired" row in the Operating Modes table.

No current *write-capable* LAN mode exists: the server is hard-limited to `127.0.0.1:8080` plus the read-only LAN view listener above, and the earlier plain-HTTP `--lan` compatibility path was removed. This future mode is a separate paired-HTTPS profile, not a convenience switch, and must:

- Require HTTPS so service workers, credentials, and browser security semantics operate on a secure origin.
- Display a short-lived pairing code locally on the server terminal.
- Pair a browser into a revocable device session; do not use a permanent URL token.

- Show bound interfaces and active paired devices at startup.
- Support revocation and session expiry from the server CLI.
- Rate-limit pairing, authentication, cloning, search, diff, and provider requests.
- Warn that guest Wi-Fi and shared classroom networks are not trusted boundaries.

Future Team Mode

Do not implement Team mode as "LAN mode with more cookies." It requires:

- External identity through OIDC or passkeys/WebAuthn.
- Per-user repository grants and operation attribution.
- Separate workspaces/worktrees for users who may edit concurrently.
- Administrator-controlled repository roots and provider credentials.
- Audit retention, session revocation, quotas, and reverse-proxy deployment.
- A decision about whether Git commits use service identity or user identity.

These concerns are intentionally outside the V2 local-first implementation.

Repository Isolation

Implemented by the server-owned catalog (M1.03, `git-vista-server::catalog`; ADR 0003). The catalog is the only path→id resolver and fails closed on anything it did not itself register.

- Configure one or more allowlisted repository roots. *(Catalog `AllowedRoots`; the clones root is always allowed, and a server-launched repo allows its own root.)*
- Canonicalize discovered paths and reject escapes through symlinks. *(Registration canonicalises the repository root and checks component-wise containment, so a `../` traversal or a symlink escaping an allowed root is rejected.)*
- Give the browser opaque repository IDs, not arbitrary filesystem paths. *(Requests select a worktree by `WorktreeId`; `GET /api/catalog` reports capabilities by id. A malformed id is a `400`, an unknown id a `404`.)*
- Resolve git-dir and common-dir through Git/gix for normal, bare, and linked worktree repositories. *(`git_vista_git::read_repo_facts` classifies each as bare / main / linked and derives the canonical root.)*
- Never follow a browser-provided path to open a file or repository. *(No endpoint accepts a path; ids resolve only against the registered set.)*
- Report capabilities without exposing absolute paths by default. *(Descriptors and the graph label carry only a base name unless `GIT_VISTA_EXPOSE_PATHS` is set. The guarantee holds on failure paths too, not only on success ones: a `WorktreeCensus::CensusFailed` carries a client-safe `reason` and a separate, flag-gated `detail`, and the full text is always written to the server's own log regardless — the flag withholds from a client, it never destroys. #657, ADR 0119.)*
- Scope operation stores and recovery refs to the canonical repository identity. *(Deferred to M1.09.)*
- Treat submodules as separate repositories with explicit opt-in traversal. *(Deferred.)*
- Never serve `.git` internals as static files.

Command Execution

- Every Git-Vista-owned spawn site uses direct argv execution; never a shell. *(Enforced by a tripwire test: ADR 0017, #144 — `git-vista-server::argv_boundary` scans both native crates and fails on any process-spawn site that is not allowlisted or does not name `git` literally. This governs Git-Vista's own spawn sites only: once invoked, `git` may run a repository hook, filter, or credential helper that itself invokes a shell — that execution is Git's, not a Git-Vista spawn site, and is addressed by the hook-policy bullet below and ADR 0025, not this tripwire.)*

- Build argv only from typed operation planners and validated domain values. *(Extended to read state: ADR 0022, #63 — a paging cursor is server-authored and HMAC-SHA256 signed. The client may echo it back but may not author it. It is validated in a fixed order — length guard, single dot, bounded base64, constant-time tag comparison, only then JSON parse — so a forged or foreign cursor is rejected before `serde_json` sees attacker-shaped bytes and before any repository walk opens. A cursor scoped to a different repository or worktree returns the generic `400`, deliberately not a distinguishing error, so probing cannot confirm that another target exists.) (Implemented for every served-repository mutation: ADR 0015/0016, #142/#143 — write handlers build a typed `GitOperation`, and `git-vista-server::planner` is the only place a mutating git argv is constructed. Proven at the API boundary by adversarial fixtures: ADR 0017, #144 — no route deserializes a raw command string or argv array, and hostile bodies die at the extractor.)*
- Pass `--` where Git supports it and validate full refnames with Git.
- Clear or explicitly set child environment variables. Disable terminal prompts, editors, pagers, and hooks where the operation semantics permit.
- Decide hook policy explicitly. Running repository hooks may execute arbitrary local code; local mode may allow them, but the UI must report that fact and Team mode should default to a restricted policy. *(Implemented: ADR 0025 (declared + disclosed) amended by ADR 0030 (enforced) — `sandbox::tier_for` decides the real tier per operation and `sandbox::hook_policy::hook_policy_for_repo` only renames that answer to the wire vocabulary, never re-derives it, so disclosure cannot drift from enforcement. `RepositoryDescriptor.hook_policy` and `SessionInfo.hook_policy` carry it to the client; `via_lan` has no bearing on the value reported (`session_hook_policy_for`, `crates/git-vista-server/src/handlers/ session.rs`) — see the ADR 0025 amendment. The client reports the per-repository value on every picker row and on the mode screen where a repository is opened (`hook_policy_disclosure.rs` + `picker.rs`, #208), with an absent value shown as "not disclosed" and styled as a warning. The separate **session**-level banner (`hook_policy_banner.rs`) also reports the real policy now (#208): its text is `hook_policy_disclosure::for_session(policy)`, an exhaustive per-variant match, not the fixed pre-#202 sentence; see ADR 0030 Consequences.)*
- Apply timeouts, cancellation, stdout/stderr limits, process-group termination, and concurrency quotas.
- Convert raw Git errors into structured safe errors; retain detailed stderr only in local protected logs with credential redaction.

Sandbox Mechanism Boundaries

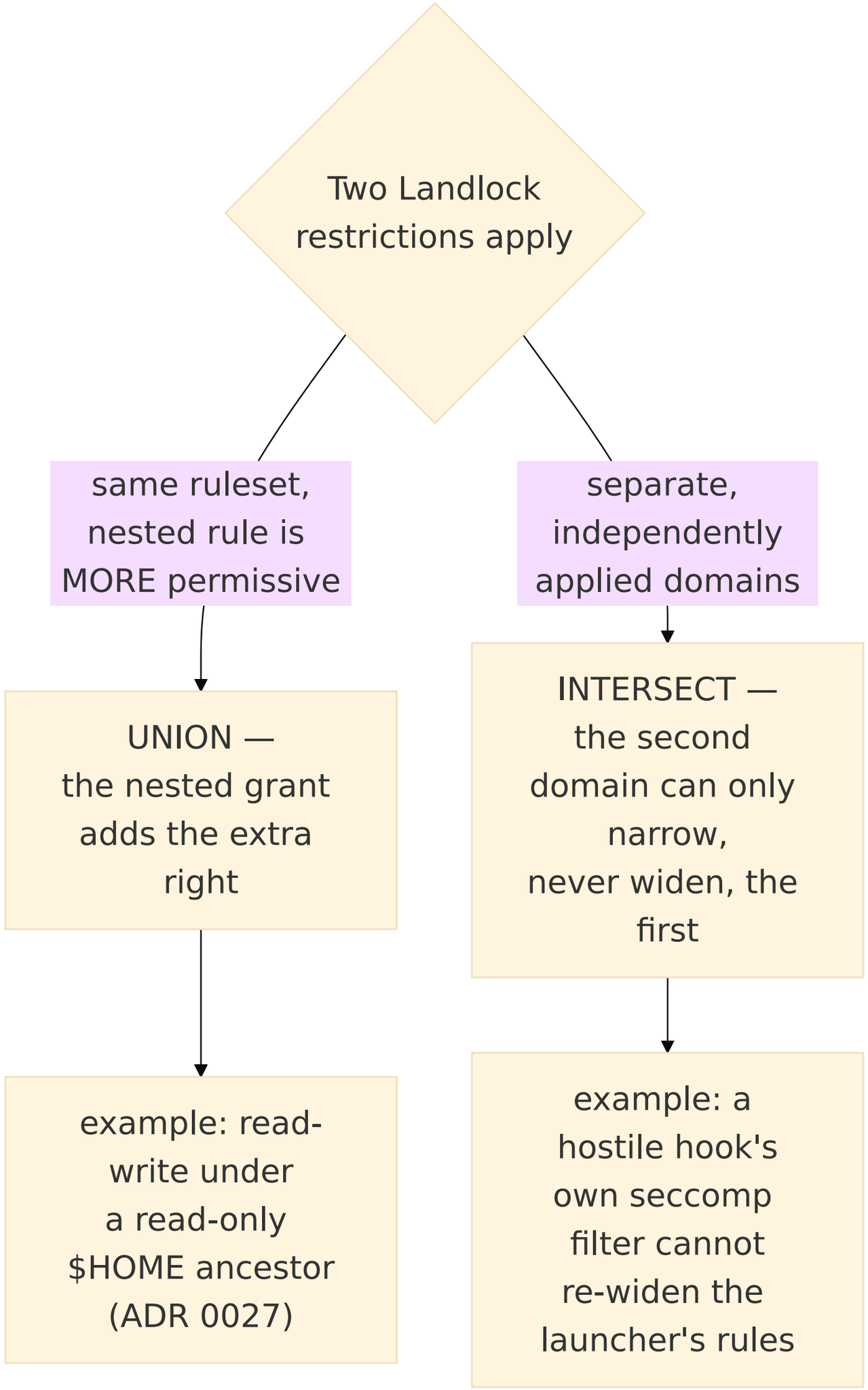
The Command Execution and Known Non-Goals sections state *what* is restricted and disclosed. The enforcing shim now ships and runs (ADR 0030); this section states what each mechanism actually covers when it does, because a boundary that is silently narrower than it sounds is worse than one stated plainly — and every limit below is a live limit, not a forecast. ADR 0027 (filesystem) and ADR 0028 (network) are the durable record behind each item below.

- **Landlock network rules authorize TCP ports, never hosts.** A single rule granting a port permits `connect()` to that port on *every* destination; the kernel's rule type carries no address field at all. The network tier's port list (ADR 0028) blocks reaching an arbitrary *local* port — a stray loopback service, a resolver, this server's own port — and cannot confine which remote host a permitted port reaches.
- **UDP and `AF_UNIX` are not mediated by these Landlock network rules at all.** Only the strict tier's network namespace blocks UDP egress, by removing network access entirely; the network tier's Landlock port rules pass UDP and Unix-domain traffic through unmediated, in either direction.
- **`AF_UNIX` in the strict tier is denied by seccomp, and by nothing else.** *Implemented* (2026-07-29) in `seccomp_filter::af_unix_rule` as an argument-scoped `EPERM` on `socket(2)` / `socketpair(2)` when the address family is `AF_UNIX`, installed for `--net-deny` (strict)

only. It is worth being explicit about why every other layer misses this, because for a period the design claimed the denial while the build did not have it (measured: both entry points succeeded inside the full strict stack, identical to the bare host, while a `ptrace` control in the same run returned `EPERM`). `LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET` mediates *connecting* to an abstract socket created outside the domain — not socket construction — and Landlock ABI 8 does not mediate **pathname** sockets at all; the IPC and network namespaces do not cover `AF_UNIX` either. Without the seccomp rule a hostile hook in the strict tier reached `/run/docker.sock`, `ssh-agent`, `gpg-agent` and the D-Bus session bus. The **network tier deliberately still permits `AF_UNIX`**: git over SSH wants an agent socket. *Implemented* (2026-07-31, ADR 0033, #188 — `Policy`'s Network branch grants the path named by `$SSH_AUTH_SOCK` read-write, alongside a narrow `known_hosts` carve-out through `secret_excludes`, described below). Measured while building that carve-out: the Landlock filesystem grant on the socket is not actually what makes it reachable on this kernel — Landlock does not mediate `connect()` to a **pathname** `AF_UNIX` socket at all (only the abstract-namespace scope above), so this tier's filter being otherwise unchanged is what genuinely leaves the socket reachable; the grant is added anyway for auditability and in case a future Landlock ABI starts mediating it (ADR 0033 §3). Proven by `af_unix_socket_denied` and `af_unix_socketpair_denied` in the escape battery, which die under `ci/mutants/M8-remove-af-unix-socket-rule.patch`. The rule's `Dword` comparison width carries its own guard: `high_bit_af_unix_denied` issues a raw `syscall(SYS_socket, AF_UNIX | 1<<32, …)` — libc's `int`-typed wrapper would truncate the hostile bits before the seccomp-visible register held them — and dies under `ci/mutants/M9-widen-af-unix-comparison.patch`.

- **A seccomp denylist keyed on bare syscall numbers does not cover the x32 ABI, and the filter's arch check does not catch it either.** x32 has no `AUDIT_ARCH` of its own: it reports `AUDIT_ARCH_X86_64` and marks itself by setting `__X32_SYSCALL_BIT` (`0x4000_0000`) in `seccomp_data.nr`. seccompiler emits one `BPF_JEQ` per bare key against the raw `nr` with `mismatch_action = Allow` as the fallthrough, so an x32-numbered call matched no key and fell through — and because the miss happens at the shared `nr` load, it voided the **entire** map at once: io_uring, `unshare`/`setns`, `seccomp` (the C1 stacking denial), `ptrace`, and the AF_UNIX rules together. Measured 2026-07-29 with hand-built cBPF of seccompiler's exact shape; the sibling i386 vector *is* closed, and closed fatally (`AUDIT_ARCH_I386` mismatch → `SECCOMP_RET_KILL_PROCESS`). Not exploitable on the development host — its kernel has `# CONFIG_X86_X32_ABI is not set`, so every x32-numbered call returns `ENOSYS` and no x32 binary can even be exec'd — but that is one kernel-config line away from live, and the sandbox neither controls nor observes the setting. `seccomp_filter::rules_with_x32_aliases` therefore inserts every key twice, bare and `__X32_SYSCALL_BIT`-set. Seccomp evaluates before the kernel's x64/x32 dispatch split, so the fix is verifiable on a host where the exploit is not: `high_bit_io_uring_denied` observes `ENOSYS` outside the sandbox and `EPERM` inside, and dies under `ci/mutants/M1-apply-seccomp-empty.patch`.
- **A Landlock `path_beneath` rule carrying a directory-only right is rejected outright when the target is a regular file — there is no partial grant.** The accepted set for a non-directory is Landlock's own `ACCESS_FILE` (`EXECUTE`, `WRITE_FILE`, `READ_FILE`, `TRUNCATE`, `IOCTL_DEV`); anything else fails the whole rule with `EINVAL`, and an empty mask fails with `ENOMSG`. Measured 2026-07-29 (ABI 8). This is why the shim masks every grant to the rights its target can carry (`gv-sandbox`'s `rights_for_target`) and why a rejected rule now **refuses the launch** rather than being discarded: a `--ro`/`--rw` entry naming a file used to grant nothing and report success, which is not failing closed — it is a weaker sandbox wearing the costume of a configured one. An unopenable path stays tolerated, because a host without `/run/resolvconf` is not a grant failure.
- **Landlock (ABI 8, this host) does not mediate metadata operations — `chmod`, `chown`, `utime`, `setxattr`, `flock`, `chdir`, `stat`, `access`.** A sandboxed process can change the mode, owner, or timestamps of a file in a tree the sandbox holds no right over at all; demonstrated first-party during round-4 probing, where a hook's `chmod 777` against a file outside every granted tree succeeded and followed the symlink. Accepted, documented non-coverage, not an unstated gap — the blast radius has not been enumerated by anyone.

- **Landlock rules bind resolved inodes, not path strings.** A name excluded from a grant is only actually withheld if the enumeration that builds the grant set resolves symlinks and matches hard-link inodes before deciding what to grant; a granted alias re-opens the excluded file by its own canonical path too. See ADR 0027 for the measured mechanism and the enumerate-and-skip algorithm this requires.
- **A Landlock domain is inherited through `fork` and preserved through `execve`, irreversibly.** This is *why* a hook or grandchild process stays under the same restriction as its parent — the property the Command Execution and hook-policy sections above depend on.
- **Rule composition differs by scope.** More-permissive nested rules *union* within one Landlock ruleset (a read-write grant nested under a read-only ancestor adds the extra right); independently applied Landlock domains *intersect* instead (stacking a second, separate restriction can only narrow what the first already granted, never widen it). Treating the two as interchangeable produces a policy that is wrong in one direction or the other.
- **`/run/docker.sock` is withheld by filesystem policy plus seccomp and descriptor discipline, not by any network rule.** uid 1000's `docker` group membership makes that socket passwordless root once reached; nothing about Landlock's network mediation is what keeps it out of reach.
- **A hook shares its uid with every other process on the host, and can act on one of them through any writable file that process watches and treats as instructions — a confused deputy over a deliberately writable object.** Neither the AF_UNIX denial nor Landlock touches this, and no unprivileged sandbox can. It is the reason "a hostile repository cannot harm you" is not purchasable, and it is written here rather than engineered against.
- **Operator trust self-propagates: `Tier::Unsandboxed` has no exclude set, so one grant is transitively a grant for every repository on the host.** The trust store is unforgeable from every *sandboxed* tier — it sits in `secret_excludes`, which outranks grants (pinned by `the_trust_store_is_withheld_even_from_a_repo_that_contains_it` in `sandbox/trust.rs`) — with one narrow, tier-gated exception since #188 (*Implemented*, ADR 0033): `Policy::ro_carveouts` names individual files — `~/.ssh/known_hosts` today, Network tier only — that bypass the exclude check (`is_or_inside_exclude`) entirely rather than competing with it, on the argument that a whole-directory exclude with no per-file override could never let git read what it legitimately needs from `~/.ssh` at all. The trust store is never such an exception: `sandbox_trust_dir()` is added to `secret_excludes` in every `Policy` constructor and has no `ro_carveouts` counterpart, so the self-propagation risk described below is unaffected by this mechanism existing. But `Unsandboxed` is the absence of the shim, so a hook in one operator-trusted repository runs with the server's full uid powers and can mint valid markers for any other repository, which the next operation then resolves as trusted. Signing markers (HMAC) does not close it: an unsandboxed hook reads any key the server can read. This is latent, not live, and deliberately so: `trust::grant` has no production caller and is compile-gated `#[cfg(test)]`, with a tripwire (`grant_is_unreachable_from_production_until_self_propagation_is_addressed`) that fails with this reasoning if the gate is removed. Wiring the operator-trust handler therefore cannot happen without confronting the choice this item records: either contain what a trusted repository's hooks can write, or disclose "trust one, trust all" in the grant UI itself.



None of the above is a defect introduced by this document; each is a true property of the underlying kernel mechanism that the sections above did not previously state.

Remote and Forge Credentials

- Prefer existing Git credential helpers and SSH agents on the Linux host.
- Never ask the browser to upload an SSH private key.
- Store forge tokens server-side using the OS keyring or an encrypted local store.
- Prefer fine-grained, short-lived provider credentials. GitHub recommends GitHub Apps over OAuth apps for tighter permissions and short-lived tokens.
- Request provider scopes per capability and show them before authorization.
- Redact URL userinfo, HTTP authorization, query tokens, and credential-helper output from logs and operation records.
- Make provider logout revoke/delete local credentials.

See GitHub's app authentication guidance.

Implemented vs. aspirational — current credential-surface census

The bullets above are the target shape; this table says which of them a real request actually gets today, so a reader doesn't have to infer "shipped" from "described."

Bullet	Status	Where
Existing Git credential helpers / SSH agent reuse	**Implemented for SSH; measured and found insufficient for HTTPS**	#188's carve-out (`sandbox/ssh_remote.rs`, ADR 0033) — `~/.ssh/known_hosts` and `$SSH_AUTH_SOCK` are reachable on every Network-tier spawn; private keys stay `EACCES`. For HTTPS, M13.01 (#582, ADR 0122) measured that an operator's own configured `credential.helper` **executes** under this sandbox but cannot read its own token store (`gh`'s config, `~/.git-credentials`, a keyring socket) — none of those are under a grant this sandbox gives out, and widening one is per-helper and open-ended. See the row below for the fix.
Git-Vista's own HTTPS credential helper	**Implemented**	ADR 0122 (#582), contained by ADR 0128 (#680) — `sandbox::network_exec::network_command_with_credential` appends (never clears) a Git-Vista-authored `credential.helper` that reads exactly one environment variable (`spawn::CREDENTIAL_TOKEN_VAR`) and touches no filesystem. The token itself never reaches argv or a URL. `POST /api/clone` performs the credentialed transfer with `--no-checkout`; after that process exits, a second Network-tier command checks out the fetched tree with hooks and filters still enabled but all three token-bearing environment names removed. `CredentialedCommand` owns the actual token and can return only token-aware redacted output. The token's *source* is `state::credential_token()`, the keyring/env/file chain (ADR 0126, #583) — see the row below.
No browser-uploaded SSH keys	**Implemented**	Structural: the browser never has a private-key upload path at all.
Private GitHub clone failure diagnostics	**Implemented**	#585 — `handlers::clone::private_repo` classifies failed GitHub HTTP transfers using token presence and Git's redacted authentication/access markers. Missing tokens, rejected tokens (401), and access refusals (403/404) have distinct messages, returned as failed clone responses and retained for retry/status polling. A 404 establishes neither repository existence nor token validity; the message leaves scope/account access unresolved. This is a stderr heuristic, not an independent GitHub API verification. Unmatched failures keep Git's redacted diagnostic, successful empty/public clones remain valid, and credentialless checkout errors bypass this classification. No token value enters the classifier or a new diagnostic channel.
`core.askpass` never runs (M1.13 finding I5)	**Implemented**	`sandbox/network_exec.rs`'s `network_command` forces `-c core.askpass=` on every Network-tier spawn; `git_cmd.rs`'s `sandboxed()` routes every `NetworkNeed::Remote` call through it, so this reaches production — every fetch, pull and push executor (`planner::fetch`, `planner::pull`, `planner::push`), not only this module's own tests.
Redact URL userinfo from logs/operation records	**Implemented**	`network_exec::redact_url_userinfo` / `redact_output` strip `user[:pass]@` from every `<scheme>://…` substring in captured stdout/stderr. `git_output_for` applies it to every `Remote`-declared call; the credential-bearing clone command applies it inside `CredentialedCommand`, and clone's credentialless checkout applies it before any output reaches the handler.
Redact HTTP `Authorization` header text and query-string tokens	**Not implemented**	Only URL userinfo is redacted. A credential surfaced as `?access_token=…` or in `Authorization: Bearer …`-shaped text would currently appear verbatim in a redacted `Output`. No test in this area exercises that shape.
Redact credential-helper output	**Implemented for the credential Git-Vista supplies; otherwise**	`CredentialedCommand` owns the exact supplied token and removes every literal occurrence from stdout and stderr, in addition to ordinary URL-userinfo redaction, before a caller can receive the `Output`. This protects clone's log and HTTP error paths even if a helper prints a bare token. A different credential unknown to Git-Vista is still covered only

Bullet	Status	Where
	URL-userinfo only	when it appears as URL userinfo; generic `Authorization` and query-token recognition remain unimplemented as stated above.
Redact command-line argv (not just captured output)	**Not implemented, primitive exists**	`network_exec::redact_args` exists as a redaction primitive for callers that log/journal "ran: git <args...>" diagnostics, but nothing currently calls it — no such diagnostic path exists yet in this crate.
OS keyring / encrypted local store for forge tokens	**Implemented, one tier not encrypted**	ADR 0126 (#583) — `token_store::resolve_token` tries, in order: the OS keyring (Linux Secret Service over D-Bus, via `keyring` v1's `Entry`), then `GIT_VISTA_GITHUB_TOKEN` / `GH_TOKEN`, then a local file at `state::token_file_path()` (`0600`, gitignored — never a tracked file). The bullet's "encrypted local store" only fully holds for the keyring tier; the tier-3 file is plaintext at rest, protected by filesystem permissions and never network-reachable, not by encryption — a real gap from the bullet's letter, recorded rather than glossed over. Every source's absence (no D-Bus session, unset env var, no file) folds into `None`, never an error — a public repository works with nothing configured. The resolved value is never logged or returned to a client: `token_store::mask_token` is the only diagnostic form, credential-bearing output is redacted by a value that owns the token, and clone removes `GIT_VISTA_GITHUB_TOKEN` / `GH_TOKEN` before spawning so an environment-backed source cannot bypass the internal helper boundary. `token_store::provenance_line` reports *which* tier answered at server startup so a stale env var shadowing a fresh keyring entry is diagnosable.
A settings surface to SET the token, plus a headless path	**Implemented**	ADR 0129 (#584) — the row above is a resolver; this is its write side, an entirely separate code path from the clone/credential-helper machinery ADR 0128 (#680) hardens. `POST /api/settings/token` (`handlers::settings::set_token`) writes to the OS keyring ONLY — the highest tier in `resolve_token`'s precedence — never the environment-variable tiers (not this process's to rewrite) or the tier-3 file (a human's manual fallback, not something the app writes to on the user's behalf when a secure store exists). `GET /api/settings/token` reports `TokenStatus { configured, masked, source }`, resolved fresh through the SAME `resolve_token()` call the credential helper itself uses — never a cached answer or a keyring-existence check — so if the keyring becomes unreadable after a save and a different tier answers instead, the status reflects that rather than a stale "configured via keyring" claim. `TokenStatus`'s own fields are plain `Option<String>` — nothing at the type level stops one from holding a raw token. The guarantee lives one level down, in the only server production response-construction path that receives the resolved secret: `token_store::token_status_of` (pure, dependency-injected) always builds `masked` from `mask_token()` and `source` from a fixed label, and is tested directly for exactly that — a real secret-shaped string in, the serialized wire bytes asserted never to contain it, for all four resolution tiers. This is a constructor-and-wire-test guarantee, narrower than a type-level seal (that would need a newtype with a private inner, not built here) and narrower than the row above's "never logged or returned" — it says nothing about, and does not depend on, whether a *transformed* token (base64, split, wrapped) could evade the credential-helper-output redaction the row above describes; that redaction is exact-literal, per ADR 0128 (#680)'s review, and this settings surface never touches that code path at all. Both routes are registered `full_routes`-only (never the LAN listener — ADR 0005's reasoning for every write/select/clone endpoint applies here too: a LAN viewer has no legitimate reason to learn whether a token is configured, masked or not) and classified in `route_authz.rs` (`GET`: session required; `POST`: session + CSRF, full write posture). The headless path needs no code here at all — `GIT_VISTA_GITHUB_TOKEN` / `GH_TOKEN` (the

Bullet	Status	Where
		row above) already work with the settings UI never opened; this row's job is only the browser-facing half.
Fine-grained, short-lived provider credentials (GitHub Apps)	**Aspirational**	No forge-app integration exists yet.
Provider scopes shown before authorization	**Aspirational**	No authorization UI exists yet.
Provider logout revokes/deletes local credentials	**Aspirational**	No provider logout or token-delete flow exists. `DELETE /api/session` revokes only the Git-Vista session and browser cookie; it neither deletes the OS-keyring entry written by `POST /api/settings/token` nor asks GitHub to revoke it. Environment-variable and file-backed token sources remain operator-managed.

The two "Not implemented" and three "Aspirational" gaps above are open scope for a future slice, not silent regressions of this one — #228's own scope explicitly reads "strip userinfo from URLs **at minimum**."

*(Fetch execution: ADR 0043, #229 — `POST /api/fetch` is the second production caller of the harness the table above describes, and the first that reads a child's output **while it is still running**. Three consequences for this section. First, redaction now covers the streaming path as well as the collected one: `git_cmd::git_streamed_for` runs every `\r`/`\n`-separated stderr record through `redact_url_userinfo` before handing it to its caller's callback, so the live sink is not a hole in the "Redact URL userinfo" row — proven against a remote that really leaks a credential-bearing URL on its own stderr, with the unredacted premise asserted in the same test (`planner::fetch_suite::a_credential_leaked_by_the_remote_never_reaches_the_operation_record`). Second, `FetchRequest` carries a **configured remote name, never a URL**, so no request can aim this server's credential helpers or SSH agent at a host of the client's choosing — see the correction below, which is where that guarantee actually comes from. Third, a fetch can now be cancelled — `POST /api/operations/{id}/cancel` SIGKILLs the direct child — and what a cancelled fetch reports about the repository is read from `refs/remotes/<remote>/*` before and after, never from git's prose. Known limitation, recorded rather than hidden: the kill reaches the direct child only, so a grandchild transport process may briefly outlive it.)*

*(Pull execution: ADR 0044, #230 — `POST /api/pull` adds **no new spawn** to this section, and that is the security-relevant decision. Its fetch half is `planner::fetch`'s own `run_fetch`, so every row of the table above applies to a pull byte-for-byte: the same `sandboxed()` chokepoint, the same forced `-c core.askpass=`, the same `redact_if_remote` on both the collected output and the live streaming records. A second `git fetch` here would have been a second surface for a credential to leak from and the first one to drift, so `planner::contract_suite` pins at source level that `planner/pull.rs` contains neither `git_streamed_for(` nor the literal `"fetch"`, and `planner::pull_suite` proves the reuse behaviourally — a pull publishes transfer progress, which only the one streaming path can produce. Like `FetchRequest`, `PullRequest` carries a **configured remote name and a branch name, never a URL**, gated by the same `RemoteConfigured` precondition. The one new exposure a pull has that a fetch does not is local, not credential-shaped: the integration half can leave the working tree half-merged, so a failed integration is aborted and the restoration is **observed** — the branch tip is re-read and `git ls-files --unmerged` listed — before the response claims the repository is back where it started.

The tier the second half runs in is part of that decision (ADR 0044 §4). A pull is the first operation in this server to run a *local* git command under a `NetworkNeed::Remote` operation, and `need` is what `tier_for` dispatches on, while `policy_for` sets `HookMode::Run` in every tier. Threading the pull's own `Remote` need into `exec_merge`/`exec_rebase` would therefore have run this repository's `post-merge` / `post-checkout` / `post-rewrite` hooks in `Tier::Network` — outbound TCP

on `DEFAULT_GIT_PORTS`, no `--unshare-net` — i.e. the identical git command would be **more** capable through `/api/pull` than through `/api/merge`, which declares `Local` and lands in `Tier::Strict`. `planner::pull::INTEGRATION_NEED` pins the integration half (and the abort, and `git ls-files --unmerged`) to `NetworkNeed::Local`, so `need` reaches exactly one call in `exec_pull`: `run_fetch`. Proved behaviourally rather than by inspection — `planner::pull_suite` runs real git hooks that report `readlink /proc/self/ns/net`, asserts the fetch half's hook shares the server's network namespace and the integration half's does not, and pins the same answer for a direct merge.)*

*(Push execution: ADR 0045, #231 — `POST /api/push` completes the remote trio and, like a pull, adds **no new spawn**: `planner::push` goes through the same `git_cmd::git_streamed_for`, so every row of the table above applies to a push unchanged — the same `sandboxed()` chokepoint, the same forced `-c core.askpass=`, the same `redact_if_remote` on both the collected output and the live streaming records. That is proven on the push path in its own right rather than assumed from fetch's proof: `planner::push_suite::a_credential_leaked_by_the_remote_never_reaches_the_push_response` stages a remote whose `pre-receive` hook prints a credential-bearing URL on stderr, asserts the **premise** by first running the same fixture through plain unsandboxed git and finding the literal secret, and then requires the server's response to carry the host but not the credential. `PushRequest` carries a branch name, an optional `set_upstream` flag and an optional `ForcePublish` — **never a URL, never a remote name, never a path**; the remote is `origin`, fixed by the handler and gated by the plan's `RemoteConfigured` precondition, so no request can aim this server's credential helpers or SSH agent at a host of the client's choosing. The `expected_remote_tip` a lease carries is a `CommitOid`, validated as forty hex characters before it can reach an argv. One limitation is worth stating here rather than only in the ADR: a push is the one operation whose effect is on a machine this server does not control, and git records `refs/remotes/<remote>/<branch>` only after the remote reports the update accepted — so a cancelled push cannot claim the remote is unchanged, and its terminal message says so instead of reassuring.)*

Which host a remote-reaching operation may contact (ADR 0047, #229 follow-up)

The paragraph above claimed the `RemoteConfigured` precondition was what kept a request from choosing the host. **It was not, and neither was the type.** Both halves were repaired; the correction is recorded here rather than edited away, because the shape of the mistake is the reusable part.

What was claimed	What was true	What holds now
`RemoteName` is a name	Its validator was "non-empty, not starting with `-`", which accepts `https://attacker.example/r.git`	`require_remote_name`: ASCII letters, digits, `.`, `-`, `_`, no leading `.`, no `..`, ≤ 100 bytes. Refuses every URL, scp-style, path and `ext::` shape. Runs from `Deserialize`, so it covers pull/push/tag too
`RemoteConfigured` gates it	`enforce_fresh` re-verified only preconditions that **held at build time**; one that already failed was skipped, on the assumption that the executor would refuse instead	`planner::refuses_when_unmet_at_build` names the one precondition with no such executor guard, and `enforce_fresh` refuses it directly

The assumption under the second row is the load-bearing lesson. It is sound for every other precondition — `git branch` refuses a name that exists, `git rebase` refuses a dirty tree — and it is false for remotes, because **git does not refuse an unknown remote; it reinterprets it as a transport target.** Verified against git 2.43.0: `git fetch ghost.git`, with no `ghost.git` remote, fetches from the *directory* `ghost.git`. The endpoint then answered `200 … already up to date`, because a fetch from an ad-hoc target moves no `refs/remotes/<remote>/*` and the before/after

diff is therefore empty — a fetch that reached somewhere it was never authorised to, reported as a no-op.

Proven by driving the real pipeline against a **live loopback listener** that must never be connected to, with a paired positive control on the same run that must connect (`planner::remote_boundary_suite`). The port is 9418 — the only unprivileged entry in `sandbox::DEFAULT_GIT_PORTS` — precisely so the sandbox's own Network-tier grant permits the connect and cannot be what makes the test pass.

Because both halves live in the shared machinery, `POST /api/pull` inherits them verbatim: `PullBranch` carries the same `RemoteName` and the same `RemoteConfigured` precondition, and the suite asserts that a pull with an unconfigured remote is refused by the gate (409) *before* `execute` is reached at all.

That assertion was written while pull's executor was still a `501` stub, to prove the refusal came from the gate rather than the stub. **With #230 landed it guards a live `git fetch`**, and the mutation that shows it says so: flipping `refuses_when_unmet_at_build`'s `RemoteConfigured` arm to `false` on this branch makes the pull answer `400 … The fetch from ‘ghost.git’ succeeded, but it has no branch ‘main’` — git really did read the in-tree directory as a transport target, through pull's own executor. The precondition arm is the only thing between a client-chosen `remote` string and that fetch.

Push reaches the same guard from a different direction, and depends on it more. `PushRequest` carries no remote field at all — `handlers::branch::push_operation` fixes it to `origin` — so `require_remote_name` has no client input to refuse on this route. **`RemoteConfigured` is the whole of the protection for a push**, and `planner::push_refuses_an_unconfigured_remote_before_reaching_its_executor` holds it down.

That test pins the refusal's *wording*, not just its status, and the mutation that justifies it is worth recording. With the `RemoteConfigured` arm flipped to `false`, a push naming an unconfigured `ghost.git` reaches git, which resolves it as a path and talks to the in-tree directory — turned away only by `! [rejected] main -> main (fetch first)`, i.e. because that fixture's local branch was not a descendant. A fast-forwardable one would have landed, and the sandbox did not intervene. **And git's own non-fast-forward rejection is also a `409`**, so asserting the status alone would have passed with the gate removed. Only `unmet_at_build`'s sentence distinguishes "we refused before contacting anything" from "the remote refused us after we contacted it" — which is the distinction this whole section exists to make.

Request Integrity

Every mutation request contains:

- Session and CSRF proof.
- Repository and worktree IDs.
- Expected repository generation.
- An idempotency key generated by the client.
- A typed operation body.
- For risky actions, a short-lived approval token from a prior plan.

The server records idempotency results for a bounded period. A retried mobile request returns the original result rather than performing the action twice.

(Generation, hash and expiry are enforced at execution time: ADR 0018, #145 — `planner::validate` refuses a tampered operation hash and an expired plan, and `planner::enforce_fresh` recomputes the repository generation (HEAD, all refs, worktree status) and re-verifies every build-time-held precondition against the live repository immediately before execution. Any drift refuses with a 409 and a client-facing reason — the TOCTOU gap fails closed. Client-supplied generation/idempotency fields arrive with the review roundtrip, M2+.)

Operation Risk Classes

Class	Examples	Required control
Read	History, diff, blame	Authentication, limits, path isolation
Local reversible	Stage, branch create	Serialized operation, generation check
History rewrite	Reset, rebase, amend	Preview, explicit confirmation, recovery ref
Worktree destructive	Clean, discard, checkout overwrite	Preview, typed file impact, safety checkpoint
Remote visible	Push, PR create/comment	Explicit target and identity confirmation
Remote destructive	Force-push, remote branch delete	Strong warning, lease/CAS, re-auth option

*(Worktree destructive — implemented: ADR 0038, #219 + #220. **Preview:** the confirmation modal names every affected path (capped at twelve, with the overflow counted, so a truncated list never understates the count) — there is no glob and no blind "discard all". This is a per-operation preview scoped to the confirm ceremony for the one discard/delete the user is about to run; it is not the broader plan-review roundtrip (client-supplied generation and idempotency fields reviewed against a live Plan, noted under Request Integrity above), which still lands with M2+. **Typed file impact:** two separate `GitOperation` variants, `DiscardTrackedPaths` and `DeleteUntrackedPaths`, each carrying an explicit `Vec<WorktreePath>` — a newtype that refuses absolute paths, `..` components, NUL bytes and option-shaped values at the wire boundary; every path is re-verified as still-tracked-dirty / still-untracked against a fresh `git status` immediately before the destructive call, and a path resolving outside the worktree through a symlink, or to a directory, is refused. **Safety checkpoint:** partial, and deliberately so. A discard is `RecoveryStrategy::RecoverableIfStaged` — the content survives as a dangling blob until the next `git gc` exactly when it was staged at some point. A delete of untracked files is `RecoveryStrategy::Irrecoverable` and **no checkpoint is possible**: the content was never in the object database, so there is nothing to checkpoint from. ADR 0038 records why manufacturing one (stash/temporary commit) was rejected, and what stands in its place — a two-step arming ceremony and copy that never says "undo", "restore" or "recover". One window stays open: `git clean` is not atomic across a multi-path pathspec, so a concurrent `git add` can produce a partial delete. That is detected without trusting git's prose for it: per ADR 0037, the delete's report is a filesystem observation (`symlink_metadata` on each requested path, before and after the spawn) rather than a parse of `git clean`'s stdout, which is gettext-translated and misreports under a non-C locale. A mismatch is a 409, never a claimed success.)*

(Lease/CAS half typed, not yet executable: ADR 0039, #227 — `GitOperation::PushBranch`'s `force: ForcePublish` makes the lease structural: `ForcePublish` has exactly two variants, `None` and `WithLease { expected_remote_tip }`, and no third, unguarded "force" variant exists anywhere in the type — a bare `--force` push cannot be constructed in Rust or deserialized off the wire. `planner::shape` turns a `WithLease` into a live `Precondition::RefAt` compare-and-swap on the remote-tracking ref (`refs/remotes/<remote>/<branch>`), bound into the plan's `operation_hash` at build time, and raises the plan's risk from `Remote` to `Destructive`.)

*(Lease/CAS half now executes: ADR 0045, #231 — the typed contract above is carried through to the argv. `planner::push::push_argv` is the **only** function in this server that builds a push command line; its `match` over `ForcePublish` has no wildcard arm, and the sole arm that emits a force flag emits `--force-with-lease=<branch>:<oid>`. So the guarantee is no longer "the type cannot name a bare force" but "no path through the executor can produce one", asserted over the whole input space (`push::no_push_argv_can_carry_a_bare_force`) and at source level for the modules around it (`contract_suite::only_planner_push_builds_a_push_argv_and_it_can_only_build_a_leased_force`). **The lease is checked twice, by two parties, against two different things, and neither subsumes the other.** Before anything is spawned, `verify_lease` compares the reviewed `expected_remote_tip` against this repository's live `refs/remotes/<remote>/<branch>`; a mismatch, a missing ref or an unreadable one all refuse (409, or 500 for "git could not run" — an unread ref is*

*evidence about nothing) and **no socket is opened and no credential is offered**. This is not redundant with the ADR 0018 staleness gate: that gate re-verifies the lease's `Precondition::RefAt` only when it held at build time, which is precisely not the stale-or-forged case. Then git's own `--force-with-lease` compares the same value against what the remote **advertises during the push** — the only check that can see a colleague's commit landing between review and submit, since the remote-tracking ref is a local cache. Both refusals are proven against a real remote's ref listing rather than a status code: `push_suite::a_lease_lost_to_a_concurrent_push_is_refused_and_the_remote_keeps_the_other_commit` and `push_suite::a_lease_tip_that_does_not_match_the_tracking_ref_never_reaches_the_remote`, the latter by an observed absence (the remote's `pre-receive` hook never fires). "Strong warning" and "re-auth option" remain open scope for the client ceremony, M2.20g #232 — the plan already carries `RiskLevel::Destructive` for the frontend to scale it from.)*

*(History rewrite, amend member — partially implemented: ADR 0040, #223. **Recovery ref:** done — a successful amend journals `ActivityKind::Amend` with the exact old→new tip pair (feeding `/api/activity`'s reset-back undo hint), and the tracked pipeline pins the pre-amend commit as a `refs/git-vista/recovery/<operation-id>` ref from the plan's `ResetRef { …, expected_tip }` (ADR 0021), so the amended-away commit survives on a real ref, not only in the reflog. The execution is compare-and-swapped on the tip the client reviewed (stale-from-the-start refuses 400 `stale_tip`; moved-while-pending refuses 409 at the ADR 0018 gate), refuses detached HEAD outright, and reports whether the amended-away commit was reachable from a remote-tracking ref as the **advisory** three-state `amended_published_commit` flag — never a block, since amending published history knowingly is legitimate; failures reach the client as typed kinds (`hook_rejected` / `signing_failed` / `stale_tip` / `other`), not stderr to sniff. **Preview and explicit confirmation:** the client ceremony is M2.19d's, still open; reset and rebase members of this row predate the plan machinery's client-review roundtrip, which lands with M2+.)*

*(Tag operations typed, not yet executable: ADR 0041, #235 — four new class members, staged contract-first exactly as ADR 0039 staged fetch/pull. **Local reversible:** `GitOperation::CreateTag` — writes a ref (and, annotated, one tag object) into the local repository only, undone by the typed `RecoveryStrategy::DeleteCreatedTag`. **Remote visible:** `PushTag`. **Remote destructive:** `DeleteRemoteTag` — a push under the hood (`git push <remote> --delete refs/tags/<name>`), so "remote branch delete" in this row now has a tag sibling; both remote-reaching variants declare `NetworkNeed::Remote` in `sandbox::network_need_for_operation`'s wildcard-free match today, binding the ADR 0036 askpass-hardening and credential-redaction tier in the type before any execution exists, and the dispatch census pins the remote set at exactly five operations. **`DeleteLocalTag` ranks `Destructive` despite never leaving the machine** — the "Local reversible" row does not fit it: `git tag -d` has no unmerged-work guard (the guard that lets `git branch -d` rank reversible), tag refs keep no reflog, and a tag can be the last ref keeping a commit alive — so it is `-D`-shaped and classified with `ForceDeleteBranch`. Its recovery control is the shape of the undo: `RecoveryStrategy::RecreateTag` carries the **unpeeled** pre-delete ref value — for an annotated tag, the tag object's own oid — so restoration via `update-ref` is byte-identical (message, tagger, GPG signature included), and `durable::recovery_oid` pins that oid under `refs/git-vista/recovery/<operation-id>`, keeping the dangling tag object reachable so `git gc` cannot prune the only exact copy while the pin exists. This is the typed **contract** only: `planner::execute` refuses all four operations with `501`, proven inert by the contract suite against real repositories and real on-disk remotes.)*

(Local tag operations now execute: ADR 0048, #238 — amends the paragraph above, which described all four tag operations as contract-only. `CreateTag` (unsigned, both kinds) and `DeleteLocalTag` execute through `POST /api/tag` and `POST /api/delete-tag`, both `full_routes`-only with the standard `SessionAndCsrf` write posture — note the deliberate split from the `GET /api/tags` listing, which stays on both listeners because it discloses only committed history. Still refused with `501`, each for its own reason: `sign: true` (M2.21e owns signing config, and silently returning an unsigned tag under a name the user believes is signed is a wrong outcome they cannot see), and `DeleteRemoteTag` / `PushTag` (the first tag code that opens a socket with

credentials on it, so it earns a review of its own — both still declare `NetworkNeed::Remote`, and the local pair declaring `Local` is what pins that nothing added here can reach a remote). ***The no-editor guarantee:*** *`git tag -a` with no message launches `core.editor`, which on a headless server is either a dead process or a request that never returns, and `git tag` has no `--no-edit` to close it after the fact — so the state is made* ***unrepresentable*** *rather than checked for. `TagAnnotation` carries a non-empty `TagMessage`, the wire DTO has no `annotated` flag (so `{annotated: true, message: null}` cannot be spelled), and the pure `create_tag_argv` emits `-a` and `-m <message>` together or neither. Proven two ways: over the argv without a spawn, and behaviourally against `.git/TAG_EDITMSG` — the witness git writes if and only if it took the editor path — with a paired positive showing a blocking editor really does hang the process the executor avoids.* ***The recovery control is now demonstrated, not asserted:*** *against a tag whose commit no branch reaches, `durable::write_recovery_ref`'s pin leaves both the tag object and that commit alive through `git gc --prune=now`, and `update-ref` at the unpeeled oid restores the tag byte-identically; the paired leg without the pin loses both. That is why `DeleteLocalTag` keeps its `Destructive` rank — the recovery exists, but it is a recovery, not the automatic guard `git branch -d` provides.)*

(Signed tag creation now attempted, not refused: #239, M2.21e — amends the paragraph above's `sign: true` clause, the last `501` this row still had. `exec_create_tag` runs a real `git tag -s` rather than refusing before any argv exists; `create_tag_argv` grows a signed arm (`-s` in place of `-a`, `-m <message>` still structurally guaranteed by the same non-empty-`TagMessage` invariant the paragraph above records). ***This does not open the AF_UNIX gap*** *the strict-tier paragraph above (§ "`AF_UNIX` in the strict tier is denied by seccomp") describes — that denial is exactly what makes signing fail on this server today, by design, not a side effect worked around here: gpg's own agent handshake is a `socket(AF_UNIX, …)` call, seccomp answers `EPERM` synchronously, and `~/.gnupg` sits in `sandbox::DEFAULT_SECRET_EXCLUDES` (the same Landlock exclude that withholds `~/.ssh`), so the dominant failure — measured on this host, not assumed — is git failing on a keyless lookup before it ever reaches the agent. Every failure lands in one of a small closed set, `SignTagFailureKind` (`no_secret_key` / `agent_unreachable` / `gpg_not_installed` / `timed_out` / `other`), reported as a typed `SignTagError` — never raw gpg stderr, which `classify_sign_failure` reads only from GnuPG's own machine-readable `[GNUPG:] …` status-fd protocol, never from translated prose.* ***The property this slice cannot compromise on:*** *a signing spawn must never hang past a bound, because the per-worktree mutation guard (`coordinator::lock`) is held across it. `run_signed_tag` wraps the spawn in a 10-second `tokio::time::timeout` with `kill_on_drop`, argued in that function's own doc comment to be belt-and-braces rather than the primary defence — the sandbox's own denials close every input-blocking path (agent, pinentry, editor, terminal prompt, stdin) before the timeout is ever needed, but no gpg-side flag can be delivered through `gpg.program` to make that a contractual guarantee (git execs it directly, no shell, so a flag-bearing program string just fails to exec) — so the bound is what makes "never hangs" true regardless. `DeleteRemoteTag`/`PushTag` are unaffected and still `501` for the reason above; opening `AF_UNIX` for either signing or the analogous ssh-agent gap (#188) remains explicitly out of scope.)*

No gesture, pressure threshold, swipe, or double-tap directly executes a destructive operation. Touch gestures may select or open a plan; final confirmation is explicit.

Worktree-destructive reporting is partly implemented (2026-08-02, ADR 0037, #284 — `planner::{observe_deletion, DeleteOutcome}`). `DeleteUntrackedPaths` has no safety checkpoint and can have none: an untracked path was never written to the object database, so there is nothing to check point to. The control that stands in for one is that the operation may never misreport itself — no decision branches on git's gettext-translated prose, the report is a filesystem observation, and the count states what this operation destroyed rather than what was requested. Preview and typed file impact land with the M2+ review roundtrip.

Clone and Network Controls

- Restrict clone schemes by operating mode and configuration.
- Block loopback, link-local, cloud metadata, and private-network destinations when arbitrary URL cloning is exposed beyond local mode, unless explicitly allowlisted.
- Apply clone size/time quotas and stream progress without buffering complete output.
- Store temporary clones under a managed root with ownership metadata and expiry. *(Implemented: ADR 0008, #121 — clones persist under `$XDG_DATA_HOME/git-vista/clones`, not a wiped-at-startup temp dir; deletion is explicit via `POST /api/delete-clone`, guarded to paths that canonicalize inside that root.)*
- Never delete a clone while an active repository handle references it. *(Implemented: ADR 0008, #121 — `delete_clone` refuses the currently open selection with `409`.)*
- Treat remote URLs as secrets because they can contain credentials.

Browser Security Headers

At minimum:

```
Content-Security-Policy: default-src 'self'; object-src 'none'; base-uri 'none';
  frame-ancestors 'none'; form-action 'self'; connect-src 'self'
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Referrer-Policy: no-referrer
X-Content-Type-Options: nosniff
Permissions-Policy: camera=(), microphone=(), geolocation=()
Cache-Control: no-store                # authenticated API responses
```

Static fingerprinted assets may be immutable. Do not apply `no-store` blindly to the PWA shell and destroy offline startup.

Implemented by `security_headers` (M1.04; ADR 0004), stamped on every response. Two deviations from the baseline above, both documented in ADR 0004: the CSP's `script-src` adds `'wasm-unsafe-eval'` (required by the WebAssembly runtime) and `'unsafe-inline'` (Trunk boots the wasm with an inline module script whose hash changes each build, and the server sets a static header), and `img-src` / `font-src` are widened to `'self' data:` / `'self'` for the inline SVG favicon and the bundled Nerd Font. `Cache-Control: no-store` is applied to API responses only; the SPA shell keeps `no-cache` so offline startup (a later PWA milestone) survives.

Data at Rest

- Operation history may expose commit messages, branches, and repository paths.
- Default to user-only filesystem permissions on the Linux host.
- Keep browser persistence minimal and clearable.
- Do not cache private diffs/file contents offline by default.
- Recovery refs and safety stashes require retention limits and visible cleanup.
- Provider tokens and device sessions must not be stored in the Git repository.

Audit and Logging

Log structured events with operation ID, repository ID, worktree ID, risk class, duration, result code, and before/after generation. Do not log request bodies by default. Local logs should be useful without exposing credentials or complete file contents.

Security Testing

- Route tests for missing/invalid session, CSRF, Origin, Host, content type, and repository access.
- Concurrency tests for clone switching, stale generations, duplicate requests, rebase/reset interleaving, and two browser tabs.
- Property/fuzz tests for ref, path, status, diff, and provider parsers.
- Tests for symlink escapes, linked worktrees, bare repositories, submodules, and repositories with hostile names/messages.
- Output-limit and timeout tests using intentionally noisy/slow child processes.
- Browser tests for cookie and service-worker update behavior.
- Dependency, license, secret, and vulnerability scanning in release CI.

Known Non-Goals

- Protecting a repository from its own Unix account owner.
- **Fully** sandboxing arbitrary Git hooks in Local mode. Enforcement has landed (ADR 0030): an untrusted repository's local operations run inside a Landlock+seccomp+bwrap tier, and the tier actually in force is disclosed — never a stronger one than what ran (ADR 0025's amendment). What "fully" still excludes, documented in Sandbox Mechanism Boundaries above: Landlock does not mediate metadata operations (`chmod`, `chown`, `utime`, `stat`, `flock`, ...) at all; the network tier confines ports, never destination hosts; and a same-uid adversary with access to a root-owned daemon socket, or to a writable file some outside process treats as instructions, is out of scope regardless of tier.
- Providing tenant isolation in V2.
- Making remote force-push universally undoable.
- Securing plain HTTP LAN mode; it should not exist as a supported write mode.

Security Release Gate

No release should claim safe remote or LAN write access until:

- Loopback is the default bind.
- SSH tunnel mode is documented and tested on iPad.
- Every mutation has session, CSRF, origin, generation, and serialization checks.
- Repository paths are allowlisted and opaque to the browser.
- Child processes have time/output limits and credential redaction.
- Recovery behavior is documented for each destructive operation.
- A `SECURITY.md` vulnerability-reporting policy exists.